

Java Programing: An Introduction to OOP



Faculty of Computer Science and Mathematics, University of Kufa, Najaf, IRAQ.

1.1 Classes and Objects

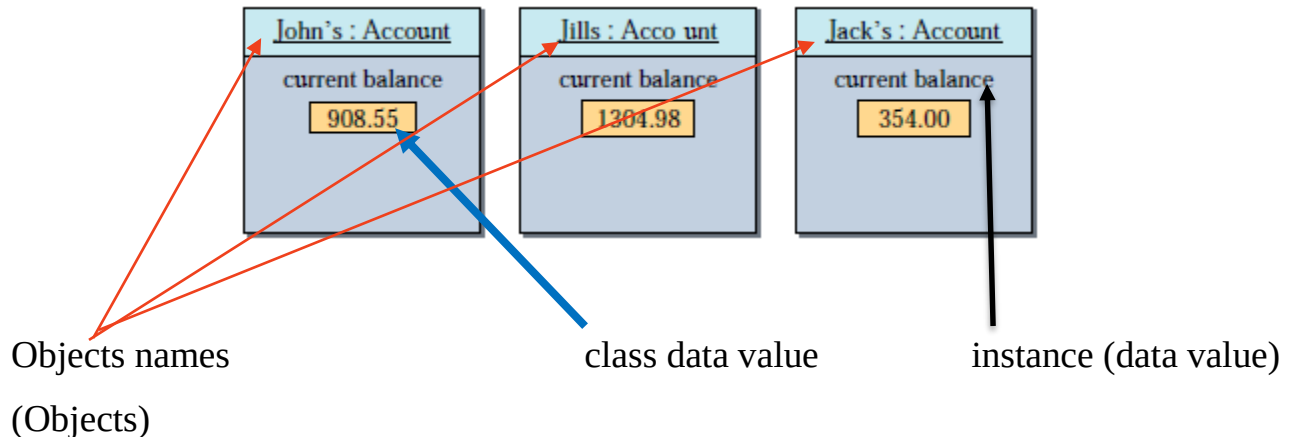
The two most important concepts in **object-oriented programming** are the **class** and the **object**. an **object** is a thing that we can imagine. A program written in object-oriented style will consist of interacting **objects**.

- For a program to keep track of **student residents** of a college, we may have many **Student, Room, and Floor objects**. For another program to keep track of **customers** and inventory for a bicycle shop, we may have **Customer, Bicycle**, and many other types of objects.
- An object is comprised of ***data and operations that manipulate these data***. For example, a **Student object** may consist of data such as name, gender, birth date, home address, phone number, and age and operations for assigning and changing these data values.
- Inside a program we write instructions to create **objects**. For the computer to be able to create an **object**, we must provide a definition, called a **class**.
- **class** is a kind of template that dictates what **objects** can and cannot do. An **object** is called an instance of a class.
-

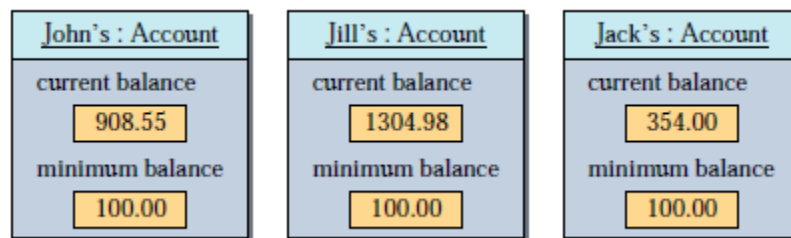
- An **object** is an **instance** of exactly **one class**.
- An instance of a **class** belongs to the class.
- Example: The two Bicycle objects **Moto-1** and **Moto-2** are instances of the **Bicycle class**.
- Once a class is defined, we can create as many instances of the class as a program requires.

A **class data value** is used to represent information shared by all *instances* or to represent collective information about the instances.

For example, an **instance** can access the **class data values** of the class to which it belongs, so every **Account object** can access the class data value as shown in the following image. مثال لحساب مصرفي حيث ان:



There are two **types of data values**: those that can change over time and those that cannot. A data value that can change is called a **variable**:



حيث ان قيم العناصر تتغير خلال الوقت .. مثال : الرصيد المصرفي لشخص اسمه جون .. يتغير المبلغ المتوفر خلال الوقت وحسب العمليات المصرفية التي يجريها على حسابه خلال فترة من الزمن

A data type that cannot changed over time is a constant, for example the opening balance at certain time or balance: على سبيل المثال وقت فتح الحساب واغلاقه / او اقل مبلغ لاجراء عملية بنكية تعتبر هذه القيمة ثابتة ولا تتغير خلال الوقت

1.2 Inheritance

The process by which one class acquires the properties (data members) and methods of another class is called **inheritance** **الوراثة**.

We group the "inheritance concept" into two categories:

- **subclass (child)** - the class that inherits from another class.
- **superclass (parent)** - the class being inherited from.

To inherit from a class, we use the **extends** keyword.

Example of inheritance is:

In the example below, the **subject** class (**subclass** or child) inherits the attributes and methods (الخصائص والعمليات) from the **Teacher** class (superclass or parent class):

```
class Teacher
{
    ... Parent Class
}

class subject extends teacher
{
    ... child class
}
```

In this example, we have a base **class Teacher** and a sub **class Example1**. Since **class Example1** *extends* the properties and work() method from base class, we need not to declare these properties and method in sub class.

```

package example1;

class Teacher
{
    String collegeName = "Kufa University";
    String levelnumber = "second year";
    String subjectname = "learn java";
    void does ()
    {
        System.out.println("2nd Year students");
    }
}

public class Example1 extends Teacher
{
    String mainSubject = "java";
    public static void main(String[] args)
    {
        Example1 obj = new Example1();
        System.out.println(obj.collegeName);
        System.out.println(obj.levelnumber);
        System.out.println(obj.subjectname);
        System.out.println(obj.mainSubject);
        obj.does();
    }
}

```

Output

```

run:
Kufa University
second year
learn java
java
2nd Year students
BUILD SUCCESSFUL (total time: 0 seconds)

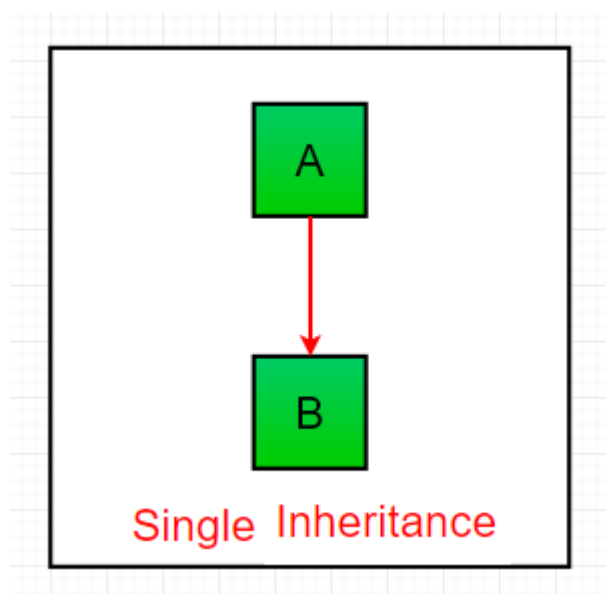
```

المثال اعلاه نلاحظ ان الكلاس الفرعي (كلاس الطفل Example1) قد استخدم الخصائص والعمليات (method) الموجودة في الكلاس الرئيسي الذي يحمل الاسم Teacher.

1.3 Types of Inheritance in Java

1) Single Inheritance

Single inheritance is easy to understand. When a class extends another one class only then we call it a **single inheritance**. The below flow diagram shows that class B extends only one class which is A. Here **A** is a **parent class** of **B** and **B** would be a **child class** of **A**.



The Above flow diagram shows that class B extends only one class which is A. Here A is a **parent class** of B and B would be a **child class** of A.

```
1 //Java program to illustrate the
2 // concept of single inheritance
3 import java.util.*;
4 import java.lang.*;
5 import java.io.*;
6
7 class one
8 {
9     public void printone()
10    {
11        System.out.println("this class one");
12    }
13 }
14
15 class two extends one
16 {
17     public void printtwo()
18     {
19         System.out.println("this class two inheritance from one");
20     }
21 }
22 // Driver class
23 public class Main
24 {
25     public static void main(String[] args)
26     {
27         two obj = new two();
28         obj.printone();
29         obj.printtwo();
30     }
31 }
32 }
33
```

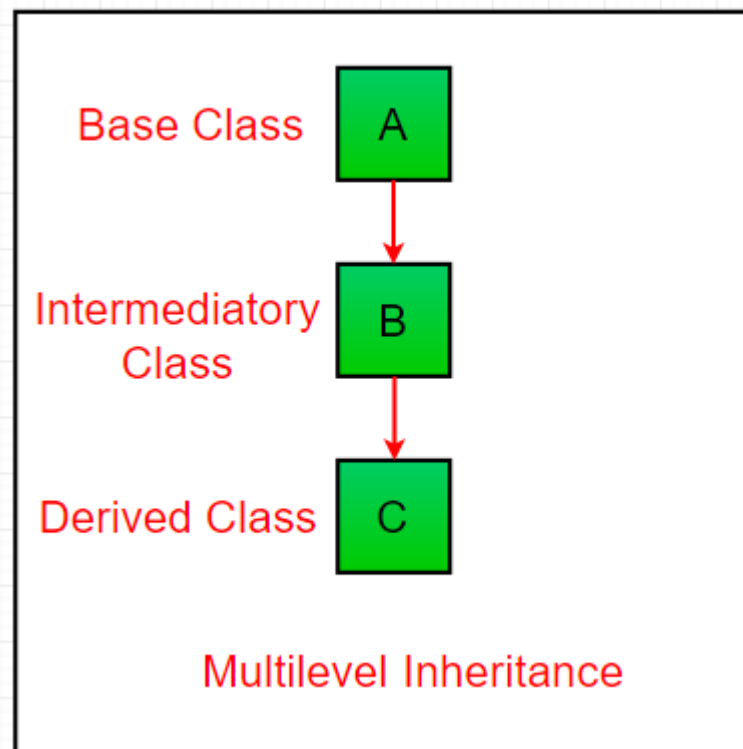
Output:

```
this class one
this class two inheritance from one
```

2) Multiple Inheritance

“**Multiple Inheritance**” refers to the concept of **one class** extending (Or inherits) more than **one base class**. The inheritance we learnt earlier had the concept of one **base class or parent**. The problem with “multiple inheritance” is that the derived class will have to manage the dependency on **two base classes**.

In below image, the class **A** serves as a base class for the derived class **B**, which in turn serves as a base class for the derived class **C**. In Java, a class cannot directly access the **grandparent’s** members.




```
1 // Java program to illustrate the
2 // concept of Multilevel inheritance
3 import java.util.*;
4 import java.lang.*;
5 import java.io.*;
6
7 class A
8 {
9     public void print_A()
10    {
11        System.out.println("Class_A");
12    }
13 }
14
15 class B extends A
16 {
17     public void print_B()
18     {
19         System.out.println("Class_B");
20     }
21 }
22
23 class C extends B
24 {
25     public void print_C()
26     {
27         System.out.println("Class_C");
28     }
29 }
30
31 // Drived class
32 public class Main
33 {
34     public static void main(String[] args)
35     {
36         C g = new C();
37         g.print_A();
38         g.print_B();
39         g.print_C();
40     }
41 }
42
```

Output

```
Class_A
Class_B
Class_C
```

3) Multilevel Inheritance

Multilevel inheritance refers to a mechanism in OO technology where one can inherit from a derived class, thereby making this derived class the base class for the new class. It refers to a child and parent class relationship where a class extends the child class. For example, **class Z** extends **class Y** and **class Y** extends **class X**.

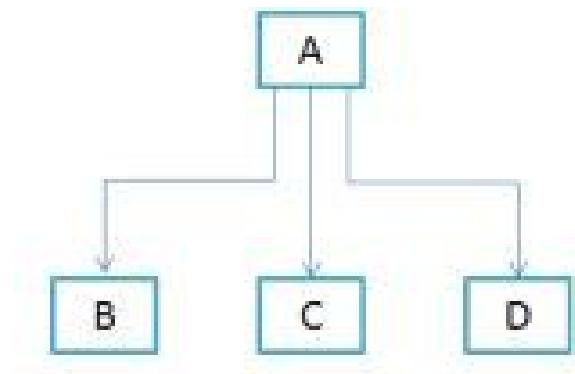
```

Class X
{
    public void methodX()
    {
        System.out.println("Class X method");
    }
}
Class Y extends X
{
    public void methodY()
    {
        System.out.println("class Y method");
    }
}
Class Z extends Y
{
    public void methodZ()
    {
        System.out.println("class Z method");
    }
    public static void main(String args[])
    {
        Z obj = new Z();
        obj.methodX(); //calling grand parent class method
        obj.methodY(); //calling parent class method
        obj.methodZ(); //calling local method
    }
}

```

4) Hierarchical Inheritance

In such kind of inheritance one class is inherited by many **sub classes**. In below example class **B**, **C** and **D** **inherits** the same class **A**. **A** is **parent class** (or **base class**) of **B**, **C** & **D**.



```

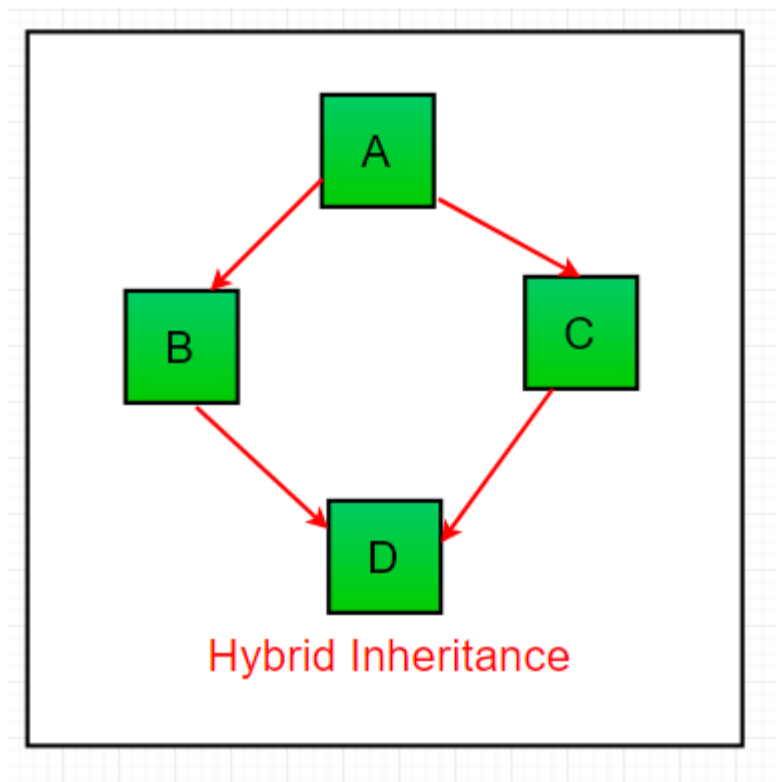
class A
{
    public void methodA()
    {
        System.out.println("method of Class A");
    }
}
class B extends A
{
    public void methodB()
    {
        System.out.println("method of Class B");
    }
}
class C extends A
{
    public void methodC()
    {
        System.out.println("method of Class C");
    }
}
  
```

```

class D extends A
{
    public void methodD()
    {
        System.out.println("method of Class D");
    }
}
class JavaExample
{
    public static void main(String args[])
    {
        B obj1 = new B();
        C obj2 = new C();
        D obj3 = new D();
        //All classes can access the method of class A
        obj1.methodA();
        obj2.methodA();
        obj3.methodA();
    }
}
  
```

5) Hybrid Inheritance

In simple terms you can say that Hybrid inheritance is a combination of **Single** and **Multiple inheritance**. A hybrid inheritance can be achieved in the java in a same way as multiple inheritance. By using **interfaces**, you can have multiple as well as **hybrid inheritance** in Java.



Note:

java doesn't support multiple inheritance with classes, the hybrid inheritance is also not possible with classes. In java, we can achieve hybrid inheritance only through **interfaces**.

```
class C
{
    public void disp()
    {
        System.out.println("C");
    }
}

class A extends C
{
    public void disp()
    {
        System.out.println("A");
    }
}

class B extends C
{
    public void disp()
    {
        System.out.println("B");
    }
}
```

```
class D extends A
{
    public void disp()
    {
        System.out.println("D");
    }
    public static void main(String args[]){

        D obj = new D();
        obj.disp();
    }
}
```

Output:

D